Fundamentals in AI and ML

PROJECT REPORT

Name: Rishav Raj

Registration Number: 25BCE11039

Branch: B.Tech CSE (Core)

Year: First Year

Course Code: CSA2001

Date of submission: 31st, March 2026

1. Project Title

House Price Predictor using Machine Learning (Linear Regression)

2. Problem Overview

Real-estate pricing decisions are often made using informal estimates, local opinions, or incomplete listing information. This creates inconsistency and can lead to poor financial decisions for buyers, sellers, and small property dealers. The problem selected for this project is to estimate a house price from structured property attributes using a reproducible machine learning approach.

The project predicts the expected market value of a house/flat based on features such as:

- Area (carpet area in square feet)
- BHK
- Bathrooms
- Resale status
- Location
- Floor details
- Furnishing status

3. Why This Problem Matters

Property valuation has practical value in daily life and financial planning. A model that produces data-driven estimates can:

- Reduce dependence on guesswork
- Provide a quick first-level benchmark price
- Help buyers compare options more objectively
- Help sellers set a more realistic listing range

From an academic perspective, this problem is also a strong introduction to supervised learning because it includes both numerical and categorical features, data preprocessing needs, and end-to-end deployment steps (train, save, and predict).

4. Project Objectives

- Build a regression-based ML model for house price estimation
- Handle mixed feature types (numerical + categorical)
- Store the trained model for reuse
- Provide a simple CLI interface for interactive predictions
- Keep the solution understandable and reproducible for students

5. Scope and Boundaries

In scope:

- Offline training on a prepared CSV dataset
- Feature preprocessing and model training
- Price prediction using user input from command line

Out of scope:

- Web or mobile UI
- Live market feeds and dynamic updates
- Legal/financial advisory recommendations
- Hyper-advanced ensemble models or deep learning pipelines

6. Dataset and Preparation

The workflow documented in the project is based on a Kaggle housing dataset. A cleaned file is used as dataset/dataset.csv for training.

Expected training columns:

- Target: Price
- Numeric features: BHK, Area, Is_Resale, Bathrooms
- Categorical features: Location, Floor, Furnishing

Data preparation strategy used in project notes includes:

- Removing rows with invalid or missing values
- Converting human-readable price strings into numeric amounts
- Extracting useful values (for example BHK and area)
- Selecting only relevant columns for modeling

7. Approach and Methodology

The project uses a supervised regression pipeline implemented in train.py.

7.1 Model Design

- Algorithm: Linear Regression
- Reason: Simple baseline, interpretable behavior, low training complexity, and suitable for academic demonstration

7.2 Preprocessing Strategy

- Numerical columns are passed through without scaling
- Categorical columns are transformed using OneHotEncoder

- handle_unknown='ignore' is enabled so unseen categories at prediction time do not crash the model

7.3 Pipeline Integration

A scikit-learn Pipeline is used to combine preprocessing and regression in one object. This ensures consistent behavior between training and inference.

7.4 Model Persistence

After training, the pipeline is serialized with pickle and saved to model/model.pkl.

8. Implementation Summary

8.1 Training Script (train.py)

- Loads dataset/dataset.csv
- Splits input features and target (Price)
- Builds a ColumnTransformer for mixed features
- Trains LinearRegression inside a pipeline
- Saves trained model as model/model.pkl

8.2 Prediction Script (predictor.py)

- Loads model/model.pkl
- Takes user input through CLI prompts
- Converts required numerical entries to float
- Converts resale input (y/n) to binary value
- Forms a single-row pandas DataFrame with expected columns
- Returns formatted rupee price prediction

9. Key Decisions and Rationale

1. Chosen algorithm: Linear Regression

Reason: Good baseline for a first version and easy to explain in a student project.

2. Use of OneHotEncoder for category columns

Reason: Required to transform text fields (Location, Floor, Furnishing) into numeric vectors usable by regression.

3. Use of a unified scikit-learn Pipeline

Reason: Prevents train-predict preprocessing mismatch and keeps model artifacts portable.

4. Command-line interface first

Reason: Faster implementation and focus on model logic before UI concerns.

5. Pickle serialization

Reason: Simple and standard way to save and reload trained scikit-learn models.

10. Challenges Faced

1. Data cleaning complexity

Raw housing data includes mixed formats, missing entries, and non-uniform text patterns. Cleaning and standardization were necessary to avoid training errors.

2. Handling categorical diversity

Location and other text features can have many values. Encoding and unseen-value handling were important for reliable predictions.

3. Input reliability in CLI

Prediction quality depends on valid numeric user input. Basic validation is present, but stronger validation can further improve robustness.

4. Model simplicity vs. accuracy

Linear Regression offers interpretability and speed, but real-estate pricing may involve non-linear relationships. This is a known trade-off in the current version.

11. Results and Observations

- The project successfully trains and saves a reusable model.
- The predictor script produces an estimated property price for user-provided details.
- The model demonstrates end-to-end functionality from training to live CLI inference.

General observations:

- Predictions are sensitive to data quality and feature representation.
- Categorical feature quality (especially location granularity) strongly influences output usefulness.
- The model is effective as a baseline estimator, not as a final authoritative valuation engine.

12. Limitations

- No formal train/test evaluation metrics are printed in the current script
- No cross-validation currently implemented

- Limited error handling for invalid/non-numeric input values

- No feature engineering for advanced signals (age of property, facing, amenities, proximity scores, etc.)

- Market conditions change over time; static dataset can become outdated

13. What I Learned

This project provided practical learning across the ML workflow:

- How to transform a real-world problem into a supervised learning task

- Importance of data cleaning before model training

- How to combine preprocessing and model training with ColumnTransformer and Pipeline

- Why model persistence is essential for deployment

- How user input interfaces must align exactly with model feature schema

- The gap between a working model and a production-grade decision system

14. Future Improvements

- Add train/test split and evaluation metrics (MAE, RMSE, R2)

- Compare multiple regressors (Ridge, Lasso, Random Forest, XGBoost)

- Add stronger input validation and exception handling in predictor.py

- Introduce feature engineering and outlier handling

- Build a lightweight web UI (Flask/Streamlit) for broader usability

- Add automated tests for training and prediction scripts

15. Conclusion

The House Price Predictor project successfully demonstrates a complete machine learning baseline solution for property price estimation using mixed-feature tabular data. It addresses a

meaningful practical problem and provides a reproducible workflow from data preparation to model training and interactive prediction.

While the current system is intentionally simple, it establishes a solid foundation for future enhancements in model evaluation, robustness, and user experience. The project met its core objective: creating a clear, functional, and educational end-to-end ML application.